

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №2
по дисциплине «Методы тестирования
программного обеспечения»
«Тестирование ПО методом белого и черного ящика»

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Изучение методов тестирования	4
1.1 Метод «черного ящика»	4
1.1.1 Классы эквивалентности	4
1.1.2 Анализ граничных значений	5
1.1.3 Причинно-следственная диаграмма	5
1.2 Метод «белого ящика»	6
2 Тестирование методом «черного ящика»	8
2.1 Описание программы	8
2.1.1 Постановка задачи	8
2.1.2 Спецификация	8
2.1.3 Блок-схема	9
2.1.4 Разбиение на классы эквивалентности	10
2.1.5 Анализ граничных значений	11
2.1.6 Причинно-следственная диаграмма	12
2.1.7 Результаты тестирования методом «черного ящика»	15
3 Тестирование методом «белого ящика»	15
3.1 Описание программы	15
3.1.1 Постановка задачи	15
3.1.2 Спецификация	15
3.1.3 Блок-схема	16
3.1.4 Покрытие операторов	18
3.1.5 Покрытие решений	18
3.1.6 Покрытие условий	19
3.1.7 Покрытие решений и условий	20
3.1.8 Комбинаторное покрытие условий	20
3.1.9 Результаты тестирования методом «белого ящика»	20
Заключение	21
Список использованной литературы	22

Введение

Модульное тестирование — это процесс тестирования отдельных блоков, подпрограмм, классов или процедур, образующих крупную программу. Другими словами, прежде чем тестировать программу в целом, необходимо сосредоточить внимание на ее меньших по размеру компонентах. Цель модульного тестирования — сравнение функций, реализуемых модулем, со спецификациями, описывающими его функциональные или интерфейсные характеристики.

В рамках данной работы требуется изучить методологии тестирования «черного ящика» и «белого ящика». После этого необходимо протестировать две программы, разработанные другими программистами, методами «черного» и «белого ящика».

1 Изучение методов тестирования

1.1 Метод «черного ящика»

В соответствии с этим методом программа рассматривается как «черный ящик», внутреннее поведение и структура которого не имеют никакого значения. Вместо этого все внимание фокусируется на выяснении обстоятельств, при которых поведение программы не соответствует спецификации.

При таком подходе тестовые данные выбираются исключительно на основе спецификаций требований, то есть без каких-либо знаний о внутренней структуре программы.

Основными техниками являются:

1. Классы эквивалентности.
2. Граничные значения.
3. Причинно-следственная диаграмма.

1.1.1 Классы эквивалентности

Определение классов эквивалентности сводится к последовательному рассмотрению каждого из входных условий и разбиению его на несколько групп. Эти группы делятся на:

- Допустимые классы эквивалентности - значения, которые программа должна корректно обработать.
- Недопустимы классы эквивалентности - значения, которые должны вызывать ошибки или некорректное поведение.

После определения классов составляются тесты. Этот процесс состоит из нескольких этапов:

- Назначение каждому классу эквивалентности уникального номера.
- Запись новых тестов, охватывающих как можно большее количество оставшихся неохваченными допустимых классов эквивалентности, пока не будут покрыты все допустимые классы.
- Запись новых тестов, каждый из которых охватывает один и только один из оставшихся неохваченными недопустимых классов эквивалентности, пока не будут покрыты все недопустимые классы.

1.1.2 Анализ граничных значений

Граничные условия — это ситуации, возникающие в области граничных значений входных и выходных классов эквивалентности. Анализ граничных значений отличается от методики разбиения на классы эквивалентности в двух отношениях:

- Необходимо выбирать именно такой элемент или элементы, которые обеспечивают тестирование каждой границы класса.
- При создании тестов внимание фокусируется не только на входных условиях, но и на пространстве результатов.

1.1.3 Причинно-следственная диаграмма

Тестирование всех комбинаций входных условий практически невозможно из-за их огромного количества. Метод причинно-следственных диаграмм систематически выбирает наиболее эффективные тесты и выявляет неполноту и неоднозначность в спецификациях. Спецификация транслируется в формальный язык (причинно-следственную диаграмму), аналог логической схемы, использующий простую нотацию. Требуется понимание базовых логических операторов (AND, OR, NOT).

Процесс построения тестов:

1. Спецификация разбивается на части, с которыми легче работать.
2. В спецификации определяются причины и следствия. Причина — это отдельное входное условие или класс эквивалентности входных условий. Следствие — это выходное условие или преобразование системы. Каждой причине и каждому последствию присваивается уникальный номер.
3. Семантическое содержание спецификации анализируется и преобразуется в булев граф, связывающий причины и следствия (причинно-следственную диаграмму).
4. Диаграмма снабжается примечаниями, задающими ограничения и описывающими комбинации причин и следствий, реализация которых невозможна из-за ограничений.
5. Путем методичного прослеживания состояний условий диаграмма преобразуется в таблицу решений с ограниченными входами. Каждый столбец таблицы решений соответствует тесту.

6. Столбцы таблицы решений преобразуются в тесты.

Базовая нотация причинно-следственных диаграмм представлена на рисунке 1. Каждый узел диаграммы может находиться в двух состояниях: 0 или 1, где 0 представляет состояние «отсутствует», а 1 - «присутствует».

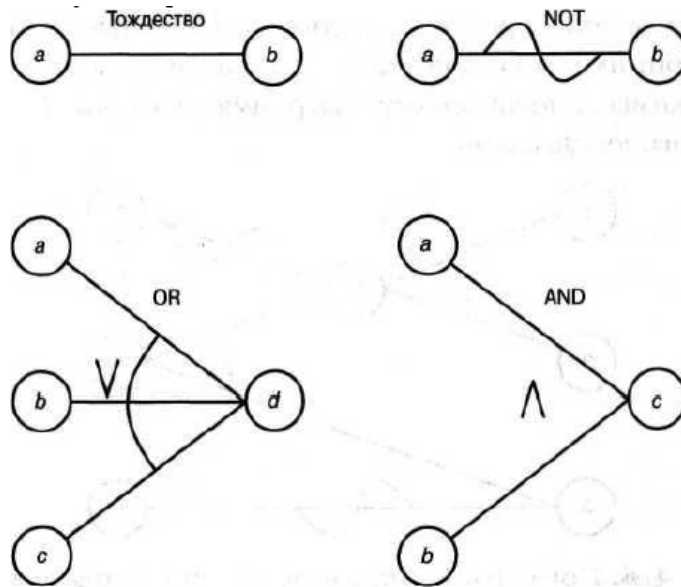


Рис. 1. Базовая нотация, используемая в причинно-следственных диаграммах

- Функция тождество устанавливает, что если a равно 1, то и b равно 1; в противном случае b равно 0.
- Функция not устанавливает, что если a равно 1, то b равно 0; в противном случае b равно 1.
- Функция or устанавливает, что если a , или b , или c равно 1, то d равно 1; в противном случае d равно 0.
- Функция and устанавливает, что если и a , и b равно 1, то c равно 1; в противном случае c равно 0.

Применение этих техник позволяет создать набор тестов, который максимально полно охватывает возможные сценарии использования программы, увеличивая вероятность обнаружения ошибок.

1.2 Метод «белого ящика»

В стратегии тестирования методом белого ящика дана внутренняя структура программы, представленная в виде блок-схемы. Задача тестирования в этом подходе заключается в том, чтобы охватить тестами

все возможные пути выполнения программы. Для облегчения и систематизации процесса применяются различные критерии:

1. Покрытие операторов.

Тесты создаются таким образом, чтобы каждый оператор программы исполнялся хотя бы один раз. Это необходимое, но недостаточное условие, так как он слабо эффективен в обнаружении сложных ошибок.

2. Покрытие решений.

В соответствии с этим критерием необходимо составить такой набор тестов, при котором каждое условие в программе примет как истинное значение, так и ложное. Этот подход лучше, чем простое покрытие операторов, так как проверяет логику переходов, но он не всегда достаточен, особенно если в программе используются составные условия.

3. Покрытие условий.

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное выражение в точке ветвления, принимало каждое из двух возможных логических значений, true и false, по крайней мере один раз.

4. Покрытие решений и условий.

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз.

5. Комбинаторное покрытие условий.

Этот критерий требует создания такого количества тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз. Это самый мощный критерий, который позволяет выявить максимум ошибок. Его основной недостаток - экспоненциальный рост числа тестов по мере увеличения количества условий.

2 Тестирование методом «черного ящика»

2.1 Описание программы

2.1.1 Постановка задачи

Автор: Черепанова Надежда

Название программы: проверка числа на простоту

Дано: a – натуральное число

Требуется: проверить, является ли число простым, и вывести соответствующий результат:

- «Составное число» – если число имеет больше двух натуральных делителей;
- «Простое число» – если число делится на 1 и на самого себя;
- «Error» – если число a не удовлетворяет ограничениям.

Ограничения: $0 < a \leq 100000$

2.1.2 Спецификация

Входные данные	Результат	Реакция программы
65	Вход: 65 Результат: Составное число	Вывод на экран результата. Завершение программы.
17	Вход: 17 Результат: Простое число	Вывод на экран результата. Завершение программы.
100001	Вход: 100001 Результат: Error. Число не удовлетворяет ограничениям	Вывод на экран ошибки. Завершение программы.
-15	Вход: -15 Результат: Error. Число не удовлетворяет ограничениям	Вывод на экран ошибки. Завершение программы.

Рис. 2. Спецификация

2.1.3 Блок-схема

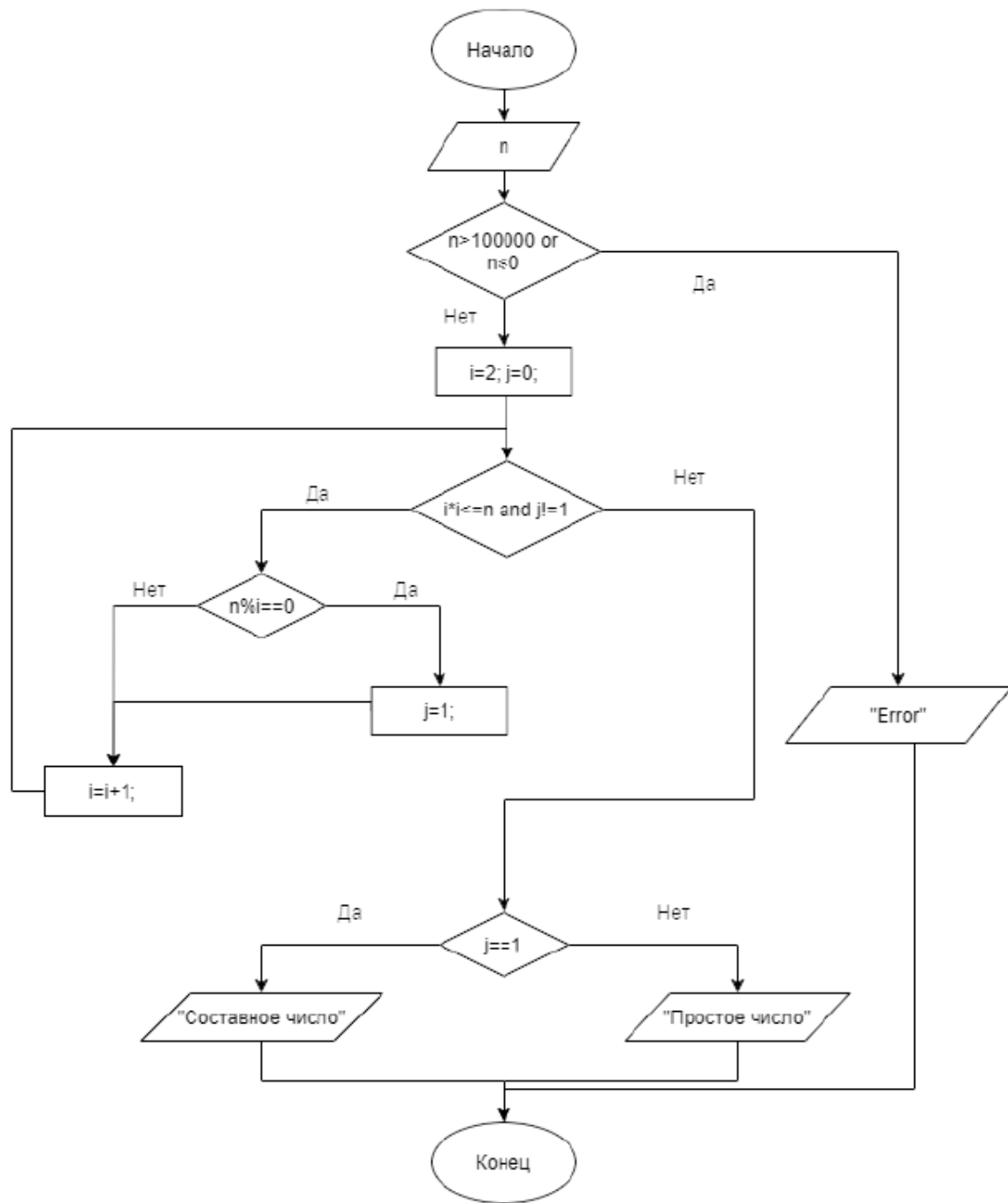


Рис. 3. Блок-схема

2.1.4 Разбиение на классы эквивалентности

Исходя из ограничений для программы (натуральное число a в диапазоне $0 < a \leq 100000$) составлено разбиение на классы эквивалентности, представленное в таблице 1.

Таблица 1. Классы эквивалентности

Входные данные	Допустимые классы	Недопустимые классы
a	a - натуральное число (1)	a - не натуральное (3)
		a - не число (4)
	$0 < a \leq 100000$ (2)	$a > 100000$ (5)
		$a \leq 0$ (6)

В таблице 2 представлены тесты, покрывающие все классы эквивалентности.

Таблица 2. Тесты, покрывающие все классы эквивалентности

№	a	Классы эквивалентности	Ожидаемый результат	Фактический результат	Результат тестирования
1	3	1, 2	Простое число	Простое число	Пройден
2	1.2	3	Error	Простое число	Не пройден
3	abc	4	Error	Простое число	Не пройден
4	100005	5	Error	Error	Пройден
5	0	6	Error	Error	Пройден

Выводы: при тестировании недопустимых классов эквивалентности были обнаружены непройденные тесты №2 и 3.

№2: при вводе ненатурального числа, большего нуля, программа должна выдать «Error» и завершиться, однако она ищет натуральные делители, не находит их и выдает результат «Простое число».

№3: при вводе строки программа должна выдать «Error» и завершиться, однако она ищет натуральные делители, не находит их и выдает результат «Простое число».

2.1.5 Анализ граничных значений

Для входных данных определены следующие граничные значения:
 $0 < a \leq 100000$.

Для каждой из границ определим тесты, соответствующие:

- верхнему/нижнему граничному целому числу;

- целому числу, выходящему за верхнюю/нижнюю границу на единицу;
- дробному числу, на 0.001 выходящему за границу (верхнюю/нижнюю).

Для каждого класса эквивалентности определим граничные значения и проведём тестирование на этих значениях.

Таблица 3. Проверка граничных значений

№	a	Ожидаемый результат	Фактический результат	Результат тестирования
1	1	Простое число	Простое число	Пройден
2	100000	Составное число	Составное число	Пройден
3	0	Error	Error	Пройден
4	100001	Error	Error	Пройден
5	-0.0001	Error	Error	Пройден
6	100000.0001	Error	Error	Пройден

Выводы: при тестировании граничных условий программа прошла все составленные тесты корректно.

2.1.6 Причинно-следственная диаграмма

Для причинно-следственной диаграммы выделим следующие причины:

1. a - натуральное число;
2. $0 < a \leq 100000$;
3. a - не натуральное;
4. a - не число;
5. $a > 100000$;
6. $a \leq 0$.

Промежуточные причины:

11. a - натуральное число и $0 < a \leq 100000$;
12. a - не натуральное или a - не число или $a > 100000$ или $a \leq 0$.

Следствия:

7. программа выводит результат;
8. программа выводит сообщение об ошибке.

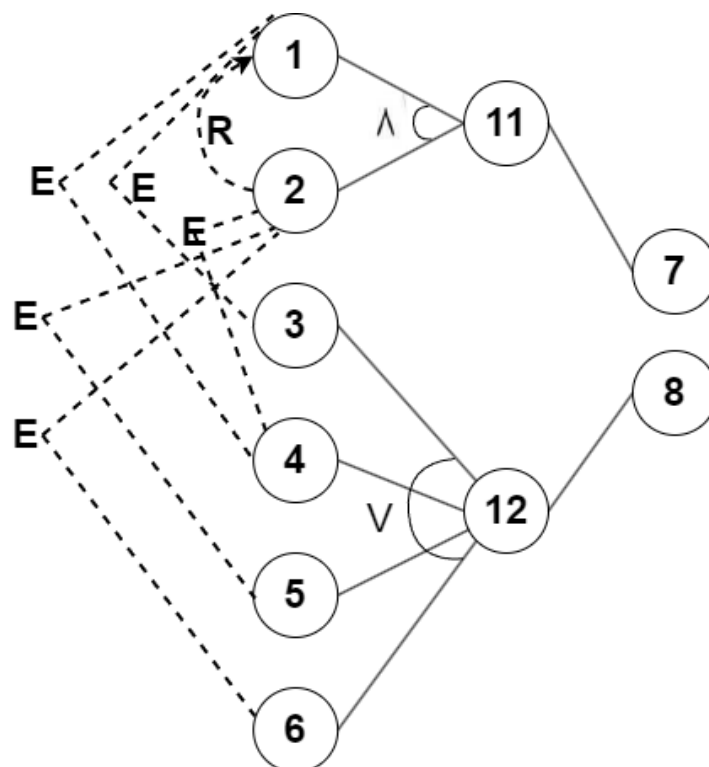


Рис. 4. Причинно-следственная диаграмма

Таблица решений на основе причинно-следственной диаграммы представлена в таблице 4.

Таблица 4. Таблица решений на основе причинно-следственной диаграммы

	1	2	3	4	5
1	1	0	0	0	0
2	1	1	0	0	0
11	1	0	0	0	0
3	0	1	0	1	1
4	0	0	1	0	0
5	0	0	0	1	0
6	0	0	0	0	1
12	0	1	1	1	1
7	1	0	0	0	0
8	0	1	1	1	1

Тесты на основе таблицы решений представлены в таблице 5

Таблица 5. Проверка граничных значений

№	a	Ожидаемый результат	Фактический результат	Результат тестирования
1	5	Простое число	Простое число	Пройден
2	2.75	Error	Простое число	Не пройден
3	сс	Error	Простое число	Не пройден
4	-56	Error	Error	Пройден
5	200000	Error	Error	Пройден

Выводы: при тестировании методом причинно-следственных диаграмм был обнаружен непройденный тест №2 и №3.

№2: При вводе ненатурального числа в пределах от 0 до 100000 программа должна выдать ошибку, однако она успешно завершается и выводит результат.

№3: При вводе нечислового значения программа должна выдать ошибку, однако она успешно завершается и выводит результат.

2.1.7 Результаты тестирования методом «черного ящика»

В результате тестирования методом черного ящика было составлено 16 тестов, из которых пройдено 12 и не пройдено 4. Ошибки, из-за которых тесты не были пройдены, связаны с отсутствием проверки корректности формата входных значений: программа выдает результат и на строковые данные, и на дробные числа, которые попадают в допустимый диапазон значений.

3 Тестирование методом «белого ящика»

3.1 Описание программы

3.1.1 Постановка задачи

Автор: Волнухина Виктория

Название программы: Жадная раскраска графа

Дано: n - целое число (количество вершин графа).

Требуется: раскрасить граф и вывести соответствующее сообщение:

- «Раскраска корректна!» — если граф успешно раскрашен и смежные вершины имеют разные цвета;
- «Ошибка! Смежные вершины имеют одинаковый цвет» — если нарушена корректность раскраски.

Ограничения:

- $4 \leq n \leq 10$

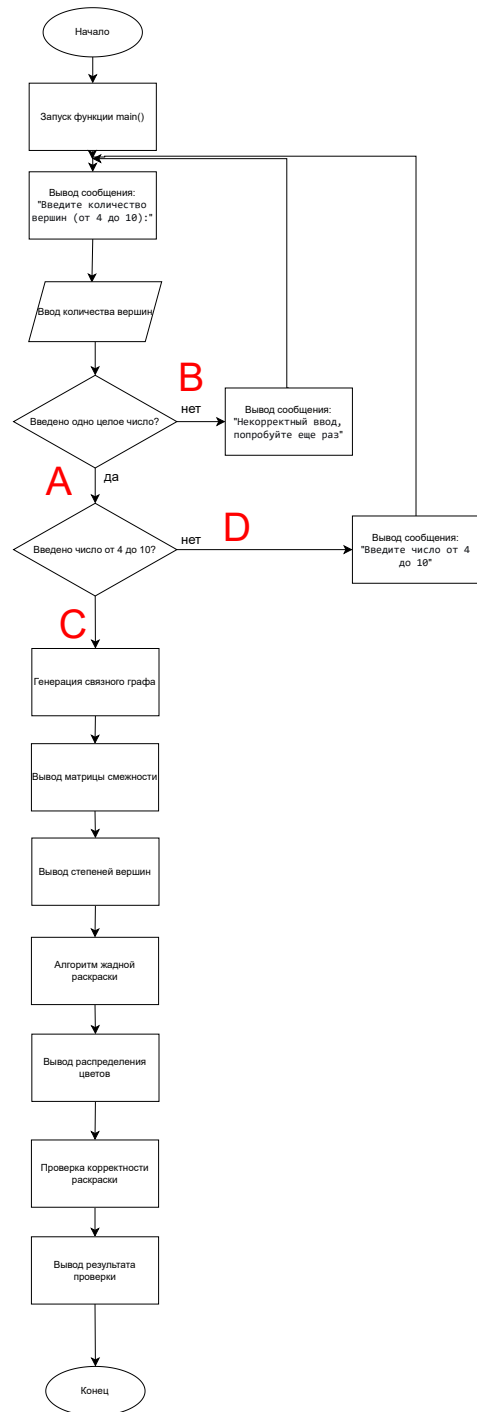
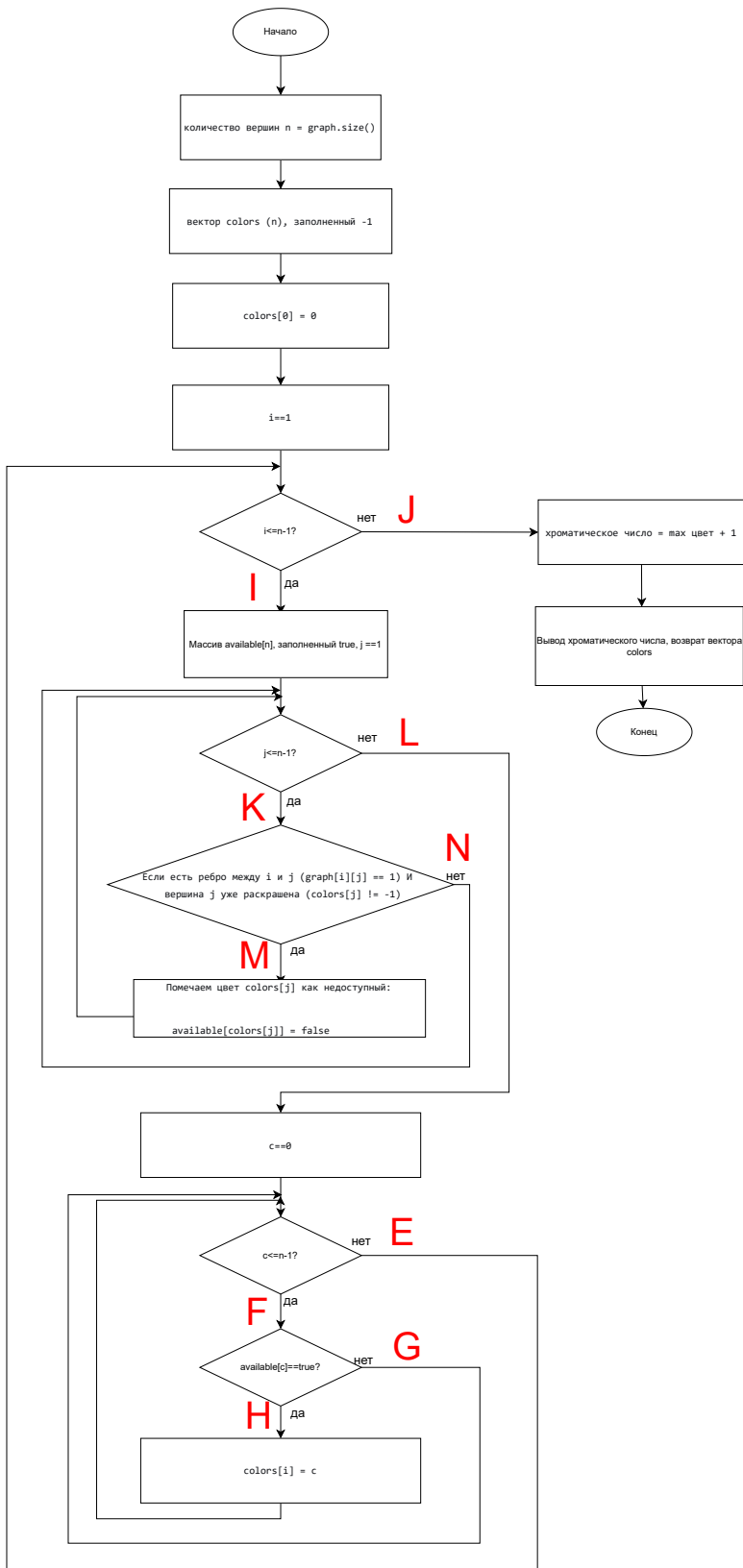
3.1.2 Спецификация

Входные данные	Ожидаемый результат	Комментарий
число, ≥ 4 и ≤ 10	Сгенерированная матрица смежности, степени вершин, распределение цветов, сообщение "Раскраска корректна!"	Граф генерируется случайным образом, применяется жадная раскраска, цвета выводятся названиями
число, < 4 или > 10	Сообщение "Некорректный ввод, попробуйте еще раз"	Повторный запрос ввода до тех пор, пока не будет введено корректное значение
нечисловое значение	Сообщение "Некорректный ввод, попробуйте еще раз"	Повторный запрос ввода до тех пор, пока не будет введено корректное значение
дополнительные проверки		
Условие	Ожидаемый результат	Комментарий
Смежные вершины с одинаковым цветом	Сообщение "Ошибка! Смежные вершины X и Y имеют одинаковый цвет!"	Проверка корректности раскраски
Все смежные вершины с разными цветами	Сообщение "Раскраска корректна! Никакие две смежные вершины не имеют одинаковый цвет"	Подтверждение успешной раскраски

Рис. 5. Спецификация

3.1.3 Блок-схема

Алгоритм жадной раскраски



Выявленные условия в программе:

1. n - целое;
2. $4 \leq n \leq 10$;
3. $i \leq n-1$;
4. $j \leq n-1$;
5. Есть ребро между i и j и вершина j уже раскрашена;
6. $c \leq n-1$;
7. `available[c] == true`;

3.1.4 Покрывание операторов

Критерием покрытия операторов является выполнение каждого оператора хотя бы один раз. Для этого каждой возможной ветви, возникающей из условных блоков, присвоим обозначения А-М.

Чтобы достичь покрытия всех операторов, предлагается воспользоваться тестом, описанном в таблице 6, который проходит по пути АСІКНКНКМЛFGFHEJ. Однако такой подход не учитывает множество возможных входных данных, что делает его недостаточно эффективным для поиска несоответствий спецификации.

Таблица 6. Покрывание операторов

№	n	Путь	Ожидаемый результат	Фактический результат	Результат тестирования
1	3	АСІКНКН КМЛFGFHEJ	Матрица смежности, "степени вершин a, b, c = 2, a=красный, b=синий, c=зеленый. Раскраска корректна!"	Матрица смежности, "степени вершин a, b, c = 2, a=красный, b=синий, c=зеленый. Раскраска корректна!"	Пройден

3.1.5 Покрывание решений

Согласно этому критерию требуется сформировать такой набор тестов, чтобы каждое условие в программе могло принимать и истинное, и ложное значения. Следовательно, помимо тестирования, ориентированного на покрытие всех операторов, необходимо добавить тесты, охватывающие все возможные ветвления. Путь, представленный в таблице 7, следующим образом покрывает решения:

Таблица 7. Покрытие решений на пути ACIKNKNKMLFGFHEJ

Условия						
1	2	3	4	5	6	7
T	T	T, F	T, F	T, F	T, F	T, F

Тесты, демонстрирующие полное покрытие решений программы, представлены в таблице 9. Данный путь покрывает решения следующим образом, как в таблице 8.

Таблица 8. Покрытие решений на пути BADACJ

Условия						
1	2	3	4	5	6	7
T, F	T, F	-	-	-	-	-

Таблица 9. Покрытие решений

№	n	Путь	Ожидаемый результат	Фактический результат	Результат тестирования
1	сс, 3, 5	BADACJ	"Некорректный ввод, попробуйте еще раз"Введите число от 4 до 10 Матрица смежности, "степени вершин a, b, c = 2, a=красный, b=синий, c=зеленый. Раскраска корректна!"	"Некорректный ввод, попробуйте еще раз"Введите число от 4 до 10 Матрица смежности, "степени вершин a, b, c = 2, a=красный, b=синий, c=зеленый. Раскраска корректна!"	Пройден

3.1.6 Покрытие условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное

выражение в точке ветвления, принимало каждое из двух логических значений, true и false, по крайней мере один раз. Уже написанные тесты покрывают каждое условие хотя бы 1 раз.

3.1.7 Покрытие решений и условий

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз.

Совокупность всех ранее написанных тестов дает покрытие условий и решений.

3.1.8 Комбинаторное покрытие условий

Этот критерий требует создания такого количества тестов, при котором каждая возможная комбинация результатов вычислений условий в каждом решении и каждая точка входа проверяются по крайней мере один раз.

Совокупность всех ранее написанных тестов дает комбинаторное покрытие условий.

3.1.9 Результаты тестирования методом «белого ящика»

В ходе тестирования методом белого ящика были разработаны 2 теста, оба были успешно пройдены. Это может одновременно указывать на корректность программы, а также на то, что разработанные тесты не учитывают какие-то случаи, в которых могла бы проявиться ошибка, например, ошибки ввода пустой строки.

Заключение

В ходе выполнения данной лабораторной работы был изучен методы модульного тестирования - «белым ящиком» и «черным ящиком».

Тестирование программы №1 по методу «черного ящика» выявило 4 непройденных теста из 16. Ошибки, из-за которых тесты не были пройдены, связаны с отсутствием проверки корректности формата входных значений: программа выдает результат и на строковые данные, и на дробные числа, что не соответствует спецификации.

При тестировании программы №2 с помощью «белого ящика» все 2 теста завершились успешно. Это означает, что скорее всего данный метод не затронул какой-то сценарий, где могла бы возникнуть ошибка.

В результате выполненной работы можно сделать вывод, что оба метода тестирования способны помочь в нахождении ошибок, но определить, какой из них эффективнее для тестирования каждой конкретной программы невозможно. Критерии метода белого ящика требуют меньшего количества тестов, но могут не выявить некоторые ошибки, выявляемые при тестировании методом черного ящика. Таким образом, лучше использовать оба метода вместе в процессе тестирования.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.